

JB Semantic Search

Technical Project Report

Nikola Zlatanovic

Report scope: semantic retrieval for scientific abstracts and Python code, with quantitative evaluation on CoSQA

Executive Summary

JB Semantic Search is a notebook-based retrieval system that connects a SentenceTransformer encoder to an embedded Weaviate vector database. The same search interface supports scientific-paper abstracts and Python code, while the main quantitative experiment evaluates natural-language-to-code retrieval on CoSQA. The project covers the complete retrieval path: loading and normalizing documents, generating dense embeddings, constructing an HNSW index, retrieving nearest neighbours, fine-tuning the encoder, and measuring ranking quality.

The primary saved notebook run indexes 20,604 code snippets and evaluates 3,921 held-out queries. The pretrained `sentence-transformers/all-MiniLM-L6-v2` baseline obtains Recall@10 of 0.3994, MRR@10 of 0.1605, and NDCG@10 of 0.2160. After one epoch of domain fine-tuning with Multiple Negatives Ranking Loss, the same evaluation reaches 0.4580, 0.1921, and 0.2539. These changes correspond to relative improvements of 14.69%, 19.65%, and 17.54%, respectively.

Two complementary experiments clarify where the gains come from. A nine-configuration HNSW sweep shows that broader graph construction and search improve candidate coverage, although the gains diminish at larger settings. A representation ablation shows that embedding complete function bodies is substantially better than embedding function names alone: the name-only variant records 0.3121 Recall@10, compared with 0.4596 for full-body embeddings in that recorded comparison. This is an absolute decrease of 0.1475, or about 32% relative to the full-body result.

The evidence supports three practical conclusions. First, even a short retrieval-specific fine-tuning run meaningfully improves code search. Second, index parameters control how effectively the database can recover neighbours already represented in the embedding space; they do not replace a suitable encoder or informative document representation. Third, function bodies carry essential semantic context that names frequently omit.

The implementation is a strong experimental prototype, but the current evidence should not be interpreted as a production benchmark. Results come from a single ordered train/test split, random seeds and full dependency versions are not pinned, the HNSW study does not record latency or memory, and repeated-run uncertainty is not reported. These limitations are made explicit throughout this report so that the measured retrieval results remain separate from claims that still require additional experiments.

Problem, Data, and Project Scope

Retrieval objective

The central task is semantic search: given a natural-language query, rank documents by meaning rather than by exact keyword overlap. For code search, the query may describe an intended operation while the relevant document is a Python function. The two texts can use different vocabulary, so lexical matching alone is insufficient. A dense encoder maps both sides into a shared vector space, and nearest-neighbour search supplies the candidate ranking.

The notebook exposes one `UnifiedSearchEngine` for several document schemas. `ArxivDocument` joins a paper title and abstract for embedding; `CodeDocument` embeds either the complete code snippet or, under an ablation flag, its extracted function name. Both are stored in type-specific Weaviate collections but retrieved through the same API. This design separates general retrieval infrastructure from document-specific serialization and display logic.

Demonstration datasets

Two demonstrations validate the general interface before the measured experiment. The paper demo loads 10,000 records from `gfissore/arxiv-abstracts-2021` and runs three qualitative queries. The code demo indexes five hand-written Python functions and runs two queries. Their saved outputs show plausible nearest neighbours, such as a quantum-computing paper for a quantum-algorithms query and an asynchronous HTTP function for the corresponding code query. These examples are useful integration checks, but they have no relevance judgements and are not counted as quantitative evidence.

CoSQA evaluation data

The main pipeline loads the query, corpus, and qrels configurations of `CoIR-Retrieval/cosqa`. It limits the query input to 20,000 items, builds the full 20,604-document corpus, and retains only queries with at least one mapped relevant document. The saved run reports 19,604 usable queries and one relevant document per query on average.

Component	Saved-run size	Role
Code corpus	20,604 documents	Search index for both training and evaluation queries
Usable queries	19,604 queries	Queries with at least one mapped relevant document
Training pairs	15,683 pairs	Query-positive-code pairs for fine-tuning
Test queries	3,921 queries	Held-out queries used for ranking metrics
Relevant documents per query	1.00 average	Ground truth used by Recall, MRR, and NDCG

The split takes the first 80% of filtered examples for training and the remaining 20% for testing. The test queries are excluded from fine-tuning, while all corpus documents remain searchable, as required for a retrieval evaluation. Because the split follows dataset order and does not shuffle, it may preserve ordering effects in the source data. The result is therefore a held-out-query evaluation, not evidence from repeated random splits.

System Architecture and Implementation

End-to-end pipeline

The system follows a direct bi-encoder retrieval architecture:

1. Load raw documents, queries, and relevance mappings.
2. Convert each corpus item into a typed `Document` object.
3. Encode document text with `all-MiniLM-L6-v2`.
4. Create a Weaviate collection with external vectorization enabled by the application (`vectorizer_config=None`).
5. Insert document properties and precomputed vectors in fixed-size batches.
6. Encode an incoming query with the same model.
7. Use Weaviate `near_vector` search to return the top ten candidates.
8. Convert returned objects back into typed documents and compute ranking metrics from document IDs.

The separation between embedding and storage is important. Weaviate does not create the vectors in this project; the `SentenceTransformer` does. This makes it possible to rebuild the same index with baseline or fine-tuned embeddings and to attribute model changes to the encoder while holding the retrieval interface constant.

Representation and schemas

The abstract `Document` interface defines the text used for embedding, the properties stored in Weaviate, and the reconstruction logic for retrieved objects. `ArxivDocument` stores title, abstract, categories, and type. `CodeDocument` stores the code text, language, and type. A type map reconstructs the appropriate object after search.

For the main configuration, `Config.func_name_embedding=False`, so the complete code field is embedded. When the flag is enabled, a regular expression extracts the first Python function name. This creates a simple and interpretable ablation: the database, model family, queries, and evaluation metric stay conceptually the same, while the amount of code context changes.

Vector database

The notebook uses embedded Weaviate and persists runtime data under `./weaviate_data`. Each evaluation resets its target collection, embeds the complete corpus, and uploads vectors in batches of 256. The query path asks for distance metadata and reports `1 - distance` as a display score. Because the collection configuration does not explicitly set a distance metric, exact score semantics depend on the Weaviate version and its default configuration; the ranking metrics depend on order rather than on the displayed score value.

The primary HNSW configuration is:

Parameter	Value
<code>ef_construction</code>	64
<code>ef</code>	64
<code>max_connections</code>	32
<code>vector_cache_max_objects</code>	1,000
<code>flat_search_cutoff</code>	4,000
<code>dynamic_ef_min</code> / <code>dynamic_ef_max</code>	100 / 500
<code>dynamic_ef_factor</code>	8

`ef_construction` and `max_connections` affect graph construction, while `ef` affects the breadth of candidate exploration at query time. Larger values can improve approximate-neighbour coverage but usually require more

construction time, query work, or memory. This project measures the quality side of that trade-off; it does not yet benchmark the resource side.

Training and Evaluation Methodology

Encoder and optimization

The encoder is `sentence-transformers/all-MiniLM-L6-v2`, selected in the notebook as a compact general-purpose sentence model with roughly 22 million parameters. The baseline is evaluated before any project-specific updates. The same model is then fine-tuned for one epoch on 15,683 query-code pairs with batch size 16 and 100 warm-up steps.

Training uses Multiple Negatives Ranking Loss. A batch contains matching query-code pairs. For each query, its paired code is the positive example, while the other positive documents in the batch act as negatives. This provides many contrastive comparisons without a separate negative-sampling pipeline. It is computationally efficient, but its quality depends on batch composition: semantically related items may become false negatives, and the notebook does not implement hard-negative mining.

After fine-tuning, the code corpus is re-embedded with the updated model and placed in a new Weaviate collection. This is required because query embeddings from a fine-tuned model must be compared with document embeddings from the same representation space.

Ranking metrics

All reported quantitative metrics use a cutoff of ten:

- **Recall@10** measures whether relevant documents are recovered among the first ten results. With one relevant document per CoSQA query, it is equivalent here to the fraction of queries whose relevant code appears in the top ten.
- **MRR@10** assigns a score of $1/\text{rank}$ when the first relevant document appears within the top ten and zero otherwise. It rewards moving the correct result toward the top.
- **NDCG@10** discounts relevant hits logarithmically by rank and normalizes against an ideal ordering. With binary relevance and one relevant item, it also emphasizes early ranking, but with a different discount curve from MRR.

Reporting all three prevents a top-ten hit from being mistaken for a consistently high first-result ranking. Recall captures coverage; MRR and NDCG capture where the successful result appears.

Comparison design

The saved baseline and fine-tuned evaluations use the same corpus, held-out queries, relevance mapping, cutoff, and primary HNSW configuration. The encoder is the intended changing component. The notebook also records two follow-up studies in its discussion: an HNSW configuration matrix and a full-body-versus-function-name representation comparison.

Those follow-up results are valuable but do not include machine-readable result files, run IDs, timing measurements, or uncertainty estimates. This report therefore presents them as notebook-recorded experimental outcomes and keeps them distinct from the output of the currently saved `RUN` cell.

Experiments and Results

Baseline versus domain fine-tuning

Metric	Pretrained baseline	Fine-tuned	Absolute change	Relative change
Recall@10	0.3994	0.4580	+0.0586	+14.69%
MRR@10	0.1605	0.1921	+0.0316	+19.65%
NDCG@10	0.2160	0.2539	+0.0379	+17.54%

Every metric improves after one epoch. The largest relative increase is in MRR@10, indicating that fine-tuning does more than recover additional relevant items within the cutoff: it also tends to move successful results closer to the first position. The absolute Recall@10 value of 0.4580 means that the relevant function appears in the first ten results for about 45.8% of the held-out queries under this run.

The result is meaningful because the candidate pool contains 20,604 code snippets rather than a small reranking set. At the same time, roughly 54.2% of test queries still miss their sole labelled relevant document in the top ten, leaving substantial room for a code-specialized encoder, better negative construction, or multi-stage retrieval.

HNSW configuration study

The notebook documents nine HNSW runs. Each cell below shows baseline to fine-tuned performance for the same recorded configuration.

ef_construction	ef	max_connections	Recall@10	MRR@10	NDCG@10
10	10	10	0.1683 -> 0.2178	0.0703 -> 0.0926	0.0932 -> 0.1218
20	20	10	0.2813 -> 0.3346	0.1088 -> 0.1343	0.1487 -> 0.1808
32	32	32	0.3578 -> 0.4139	0.1438 -> 0.1640	0.1934 -> 0.2221
32	32	128	0.3887 -> 0.4453	0.1619 -> 0.1905	0.2148 -> 0.2499
32	128	32	0.3892 -> 0.4586	0.1574 -> 0.1922	0.2113 -> 0.2540
64	64	32	0.4037 -> 0.4596	0.1662 -> 0.1913	0.2215 -> 0.2536
128	32	32	0.3948 -> 0.4611	0.1634 -> 0.1983	0.2173 -> 0.2593
128	128	32	0.4172 -> 0.4649	0.1778 -> 0.1967	0.2335 -> 0.2591
256	256	128	0.4193 -> 0.4779	0.1757 -> 0.2022	0.2322 -> 0.2663

The sweep supports two observations. First, the smallest graph/search setting is clearly insufficient for this corpus: its fine-tuned Recall@10 is 0.2178, versus 0.4779 for the largest recorded setting. Second, improvement becomes less uniform at moderate and large settings. For example, increasing only `ef` from 32 to 128 at `ef_construction=32` and `max_connections=32` raises fine-tuned Recall@10 from 0.4139 to 0.4586. In contrast, several larger configurations produce very similar NDCG@10 values near 0.259, suggesting a plateau in ranking benefit.

The notebook recommends `ef_construction=64`, `ef=64`, and `max_connections=32` as a balanced setting. Its recorded fine-tuned quality (0.4596 Recall@10, 0.1913 MRR@10, 0.2536 NDCG@10) is close to the best values while using smaller settings than the maximum configuration. “Balanced” remains a qualitative conclusion, however, because query latency, index-build time, and memory were not measured.

The saved primary run at this nominal configuration reports 0.3994 to 0.4580 Recall@10 rather than the sweep's 0.4037 to 0.4596. The gap is small, but the two records should not be merged. Without persisted run metadata and fixed seeds, it is best treated as run or environment variability rather than evidence for a precise effect.

Full function body versus function name

Representation	Recall@10	MRR@10	NDCG@10
Complete function code	0.4596	0.1913	0.2536
Function name only	0.3121	0.1245	0.1732
Absolute decrease	-0.1475	-0.0668	-0.0804

Removing the function body reduces all three metrics by roughly 32-35% relative to the full-code values. Names such as `get_data` or `set_value` are short and ambiguous, while parameters, operations, called APIs, and docstrings provide evidence needed to match a natural-language intent. The result is consistent with the representation design: faster or shorter input is not useful if it discards the semantics required for retrieval.

Discussion, Limitations, and Next Steps

What the experiments establish

The most defensible finding is the fine-tuning effect within the saved evaluation: all three metrics improve on the same held-out query set after a single epoch. The representation ablation independently shows that document content matters substantially. The HNSW matrix then demonstrates a separate systems effect: an overly shallow approximate index can hide relevant neighbours even when embeddings are unchanged.

These layers should not be conflated. Encoder training changes the geometry of the vector space. Full-body embedding changes the information supplied to the encoder. HNSW settings change how thoroughly the database explores that space. Strong indexing cannot recover semantics that were never encoded, while strong embeddings can still be underserved by a restrictive approximate search.

Methodological limitations

- **Single ordered split:** the 80/20 split is based on current dataset order, with no shuffle, stratification, or cross-validation. A different order may change the measured difficulty.
- **No validation partition:** HNSW alternatives and representation choices are compared against the available test queries. A stronger protocol would choose configurations on validation data and reserve a final test set for one evaluation.
- **No repeated runs:** random seeds are not fixed and confidence intervals are not reported. Small differences between configurations should not be treated as statistically reliable.
- **Incomplete provenance:** the notebook contains a saved primary output and manually documented follow-up tables, but no structured result artifacts linking every row to a code revision, environment, and seed.
- **Unpinned environment:** the notebook installs `weaviate-client` but assumes the remaining Colab libraries are available. The saved output shows Weaviate client 4.17.0, while other versions are not captured in a lock file.
- **Approximation without efficiency measurements:** the HNSW sweep reports ranking metrics but not query latency, throughput, index-build time, disk use, or peak memory. It cannot yet identify a measured Pareto-optimal setting.
- **Default distance configuration:** the collection does not explicitly specify its distance metric, which leaves an avoidable dependency on library defaults.
- **One labelled positive:** CoSQA's single relevant item per query simplifies evaluation, but it may count other semantically useful functions as false positives.
- **Limited training study:** only one model, one epoch count, one batch size, and one loss are evaluated. The design does not isolate whether the selected training recipe is optimal.
- **Qualitative paper demo:** the ArXiv search demonstrates generality but lacks labelled queries and quantitative evaluation.

Highest-value next experiments

The next iteration should first make the existing evidence reproducible: fix Python, NumPy, and PyTorch seeds; record package versions; save each configuration and result as JSON or CSV; and repeat the main comparison across several seeds. A shuffled train/validation/test split or official CoSQA evaluation partition should separate model selection from final reporting.

For retrieval quality, useful comparisons include a code-pretrained encoder, two or more fine-tuning durations, hard negatives mined from the baseline, and a lexical or hybrid BM25+dense baseline. For system quality, every HNSW configuration should record p50/p95 latency, queries per second, build time, and index size. An exact or high-recall reference search would distinguish embedding failure from approximate-index failure.

These are proposed experiments, not completed results. The current repository establishes a clear baseline and experimental framework on which they can be built.

Reproduction and Conclusion

How to reproduce the saved pipeline

1. Open `semantic_search_project.ipynb` in Google Colab or a local Jupyter environment.
2. For faster embedding and fine-tuning, enable a Colab T4 GPU.
3. Run all cells in order. The first cell installs `weaviate-client`; the environment must also provide PyTorch, SentenceTransformers, Hugging Face Datasets, NumPy, Matplotlib, and tqdm.
4. The notebook creates an embedded Weaviate store at `./weaviate_data` and saves the tuned model to `./finetuned_cosqa_model` during execution.
5. The `RUN` cell calls `main()`, which executes the ArXiv demo, code demo, CoSQA baseline, one-epoch fine-tuning, tuned evaluation, comparison table, and loss plot.
6. Compare the produced corpus/query counts and metrics with the values in this report before interpreting configuration changes.

On the saved workload, the notebook output shows approximately 15-16 seconds for each 20,604-document upload and 41-49 seconds for each 3,921-query evaluation in its recorded Colab session. These snippets are observational timings rather than a controlled benchmark; model encoding, training, downloads, startup, and total end-to-end runtime are not summarized in the output.

Evidence map

Report claim	Repository evidence
Model, batch size, epoch count, and HNSW defaults	<code>Config</code> notebook section
Document abstraction and vector indexing	<code>Document Schemas</code> , <code>Search Engine</code> , and <code>Weaviate DB</code> sections
Data counts and train/test split	Saved output of the <code>RUN</code> cell and <code>Data Loading</code> code
Baseline and fine-tuned metrics	Saved output of the <code>RUN</code> cell
HNSW sweep and representation ablation	Notebook <code>DISCUSSION</code> tables
Metric definitions	<code>METRICS</code> section

Conclusion

JB Semantic Search demonstrates a complete semantic-retrieval workflow rather than an isolated embedding example. It supports multiple document types, delegates dense representation to a reusable encoder, stores externally computed vectors in Weaviate, evaluates rankings against relevance judgements, and uses controlled comparisons to examine model adaptation, index search breadth, and representation content.

The primary result is clear: one epoch of CoSQA fine-tuning improves `Recall@10` from 0.3994 to 0.4580, `MRR@10` from 0.1605 to 0.1921, and `NDCG@10` from 0.2160 to 0.2539. The additional studies show that full function bodies are important and that HNSW configuration materially affects approximate retrieval coverage. The report also identifies the boundary of the evidence: production trade-offs and statistical stability still require explicit latency tests, validation discipline, repeated runs, and stronger provenance.

As a portfolio project, the repository therefore shows both implementation breadth and experimental judgement: it builds the full system, measures several aspects of retrieval quality, distinguishes model effects from index effects, and states what the current experiments do not yet prove.